

UCX is All You Need : A Universal Transform for Committing Authenticated Encryption

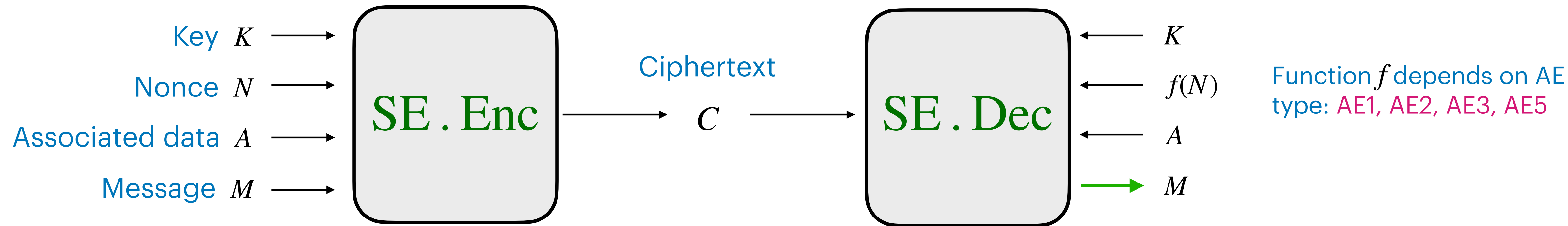
Mihir Bellare, University of California San Diego

Rishabh Ranjan, University of California San Diego

Nujud Senan, KACST

Basel Alomair, KACST, University of Washington, HUMAIN

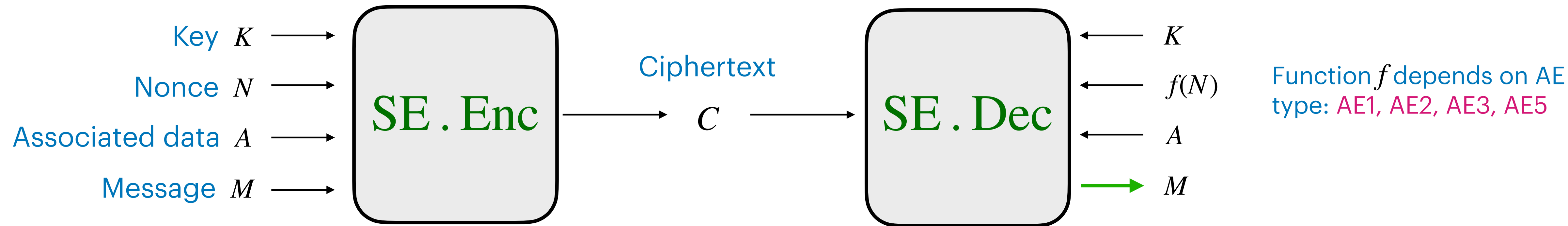
Symmetric Encryption



Classical security requirement:
AE (authenticated encryption)

- Requires:
- Privacy of message M
 - Authenticity of both M and A

Symmetric Encryption



Classical security requirement:
AE (authenticated encryption)

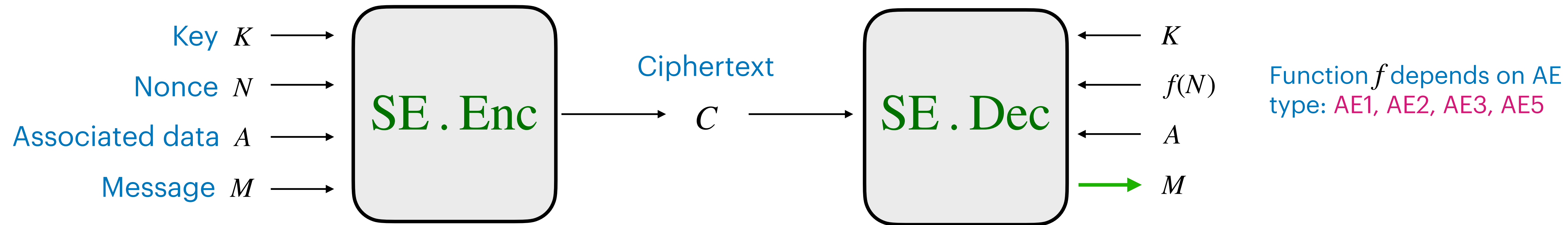
- Requires:
- Privacy of message M
 - Authenticity of both M and A

Newer security requirement:
CMT (committing security)

Requires: **Hard to find**
 $(K_1, N_1, A_1, M_1) \neq (K_2, N_2, A_2, M_2)$
such that
 $\text{SE . Enc}(K_1, N_1, A_1, M_1) = \text{SE . Enc}(K_2, N_2, A_2, M_2)$

↑
Strong definition from [BH22,CR22]

Symmetric Encryption



Classical security requirement: AE (authenticated encryption) + Newer security requirement: cE (committing security) = **cAE** committing authenticated encryption



Why should AE be committing?

Employing **non-committing AE** has led to **practical attacks** in several applications such as:

- Key rotation in Key Management Services [ADGKLS22]
- Envelope encryption used by cloud service providers [ADGKLS22]
- Message Franking used by Facebook Messenger [GLR17, DGRW18]

Why should AE be committing?

Employing **non-committing AE** has led to **practical attacks** in several applications such as:

- Key rotation in Key Management Services [ADGKLS22]
- Envelope encryption used by cloud service providers [ADGKLS22]
- Message Franking used by Facebook Messenger [GLR17, DGRW18]

Are existing AE schemes committing?

Why should AE be committing?

Employing **non-committing AE** has led to **practical attacks** in several applications such as:

- Key rotation in Key Management Services [ADGKLS22]
- Envelope encryption used by cloud service providers [ADGKLS22]
- Message Franking used by Facebook Messenger [GLR17, DGRW18]

Are existing AE schemes committing? **No**

Attacks have been given on:

- AES-GCM, XSalsa20/Poly1305, AES-GCM-SIV [GLR17]
- OCB3, AES-GCM-SIV and ChaCha20/Poly1305 [ADGKLS22]
- CCM, EAX, SIV [MLGR23]

Why should AE be committing?

Employing **non-committing AE** has led to **practical attacks** in several applications such as:

- Key rotation in Key Management Services [ADGKLS22]
- Envelope encryption used by cloud service providers [ADGKLS22]
- Message Franking used by Facebook Messenger [GLR17, DGRW18]

Are existing AE schemes committing? **No**

Attacks have been given on:

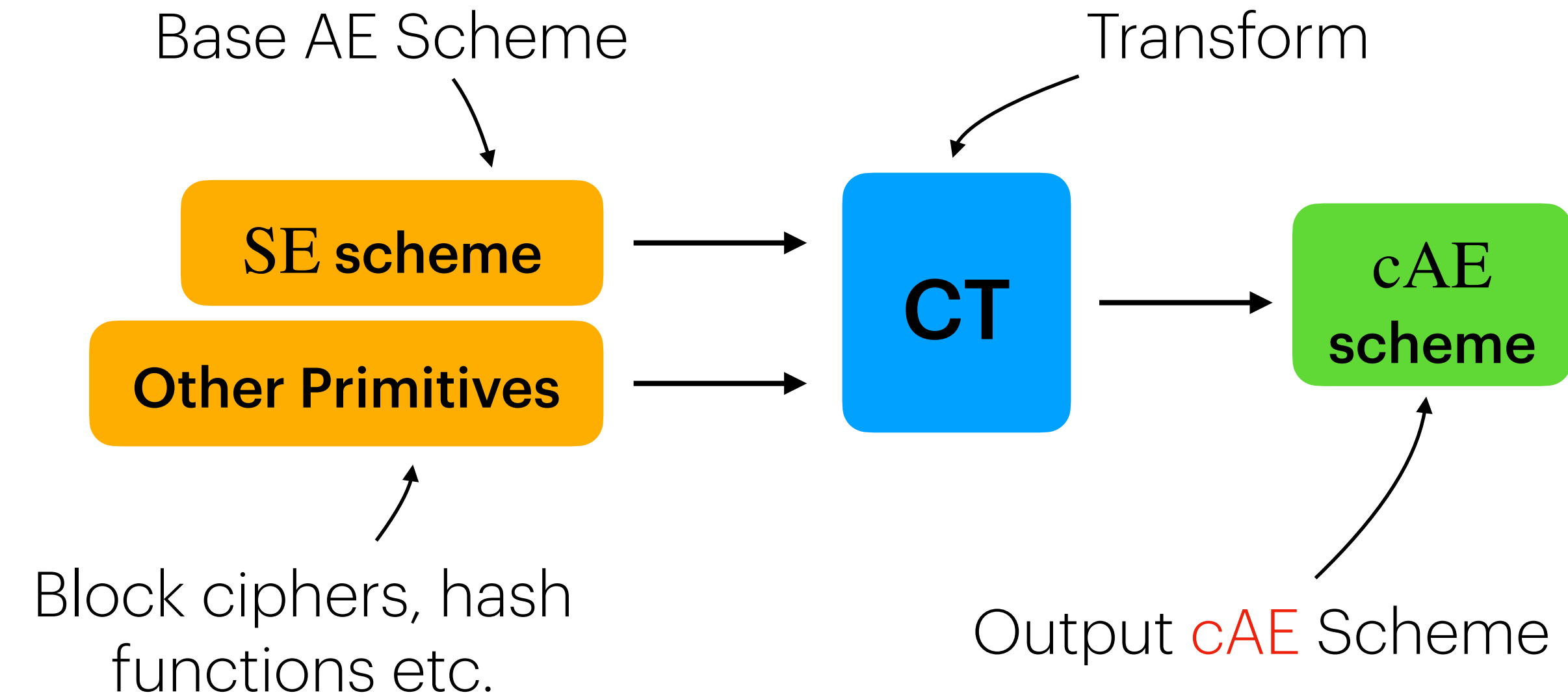
- AES-GCM, XSalsa20/Poly1305, AES-GCM-SIV [GLR17]
- OCB3, AES-GCM-SIV and ChaCha20/Poly1305 [ADGKLS22]
- CCM, EAX, SIV [MLGR23]

So we need to construct new schemes that are cAE

How are cAE schemes constructed?

Committing AE schemes are constructed via **generic transforms** that

- **Preserve AE**: If SE is AE-secure, then so is cAE
- **Add committing security**: cAE is additionally CMT-secure



How are cAE schemes constructed?

Committing AE schemes are constructed via **generic transforms** that

- **Preserve AE**: If SE is AE-secure, then so is cAE
- **Add committing security**: cAE is additionally CMT-secure

Many such transforms exist:

HtE $\circ$ UtC [BH22]

HtE $\circ$ RtC [BH22]

CTX [CR22]

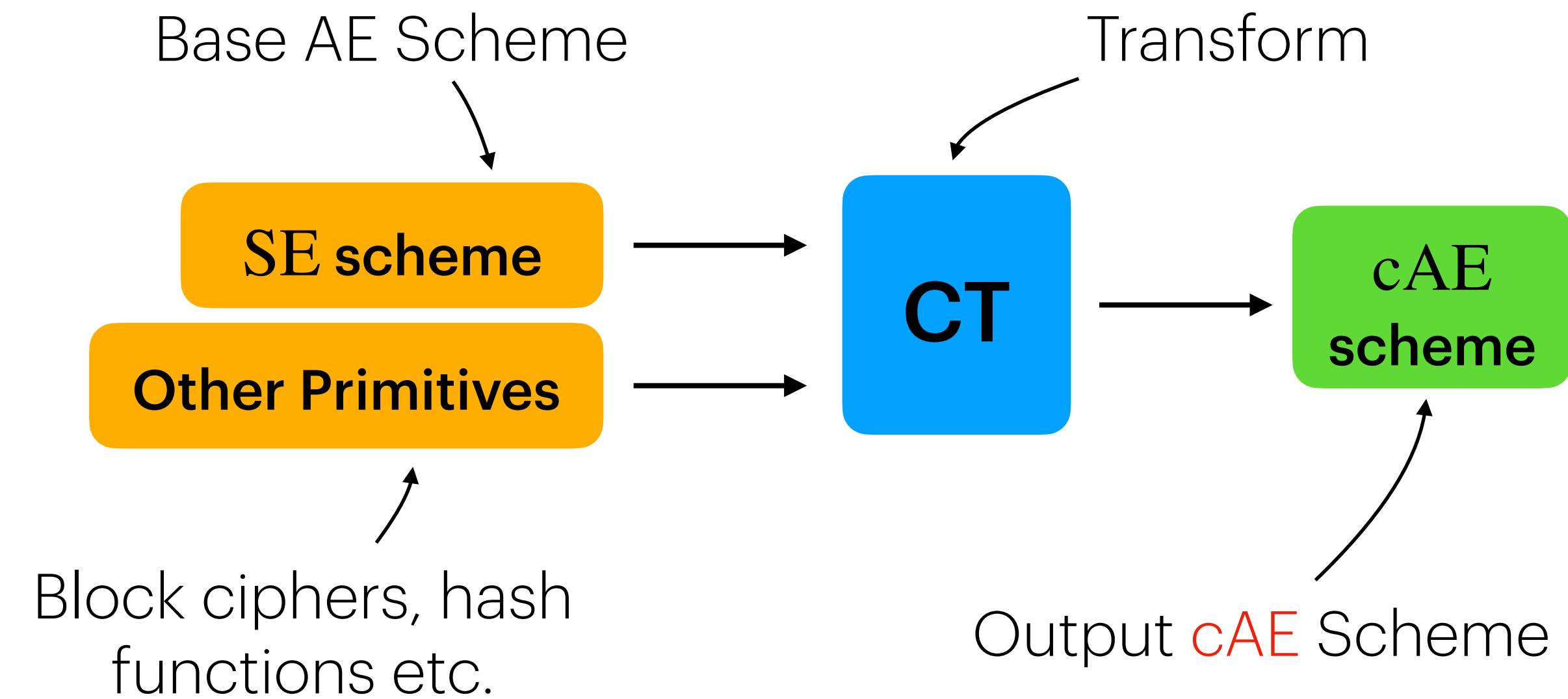
CTY [BH24]

XtH [MBHLR25]

HtAE [CK25]

AEaH [CK25]

AEtH [CK25]



How are cAE schemes constructed?

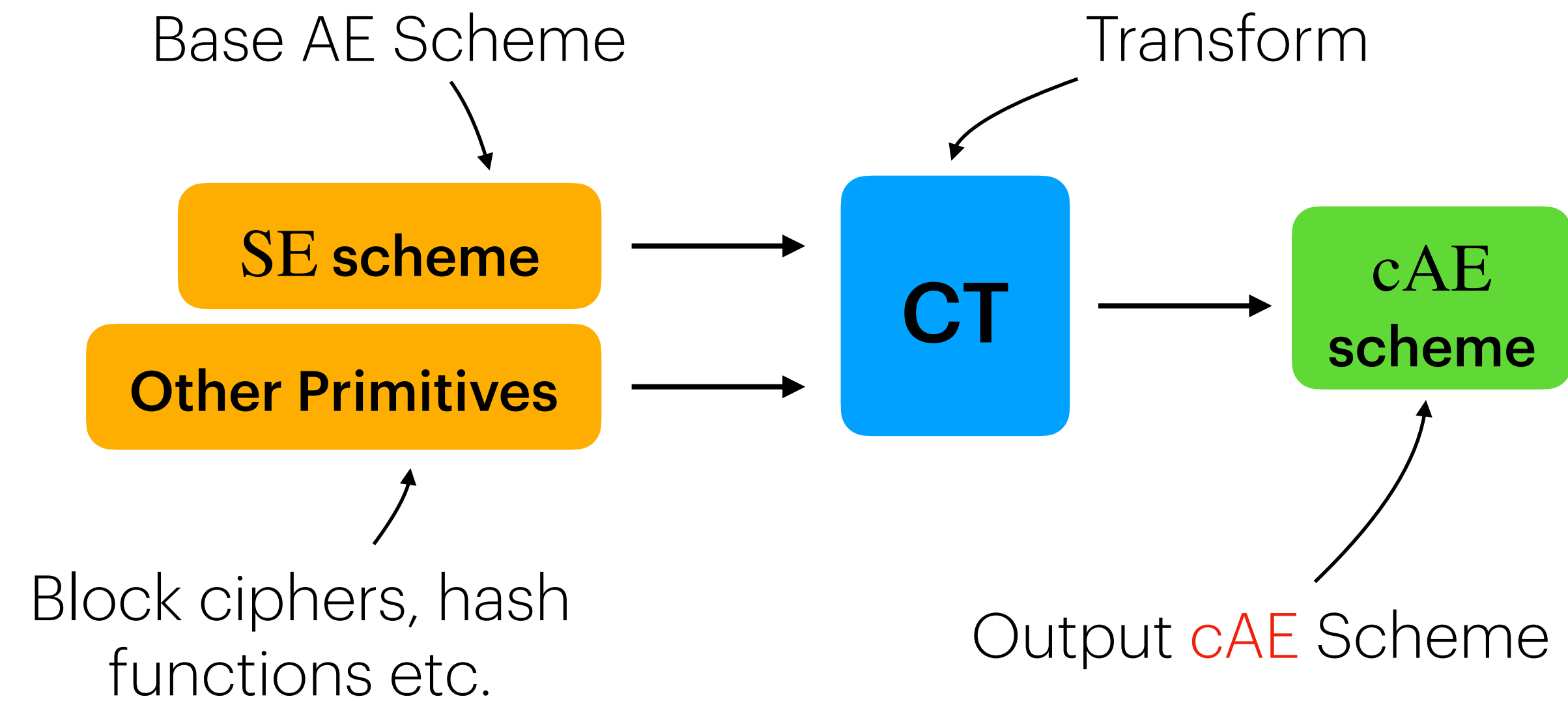
Committing AE schemes are constructed via **generic transforms** that

- **Preserve AE**: If SE is AE-secure, then so is cAE
- **Add committing security**: cAE is additionally CMT-secure

Many such transforms exist:

HtE $\circ$ UtC [BH22]
HtE $\circ$ RtC [BH22]
CTX [CR22]
CTY [BH24]
XtH [MBHLR25]
HtAE [CK25]
AEaH [CK25]
AEtH [CK25]

So what is the problem?



How are cAE schemes constructed?

Committing AE schemes are constructed via **generic transforms** that

- **Preserve AE**: If SE is AE-secure, then so is cAE
- **Add committing security**: cAE is additionally CMT-secure

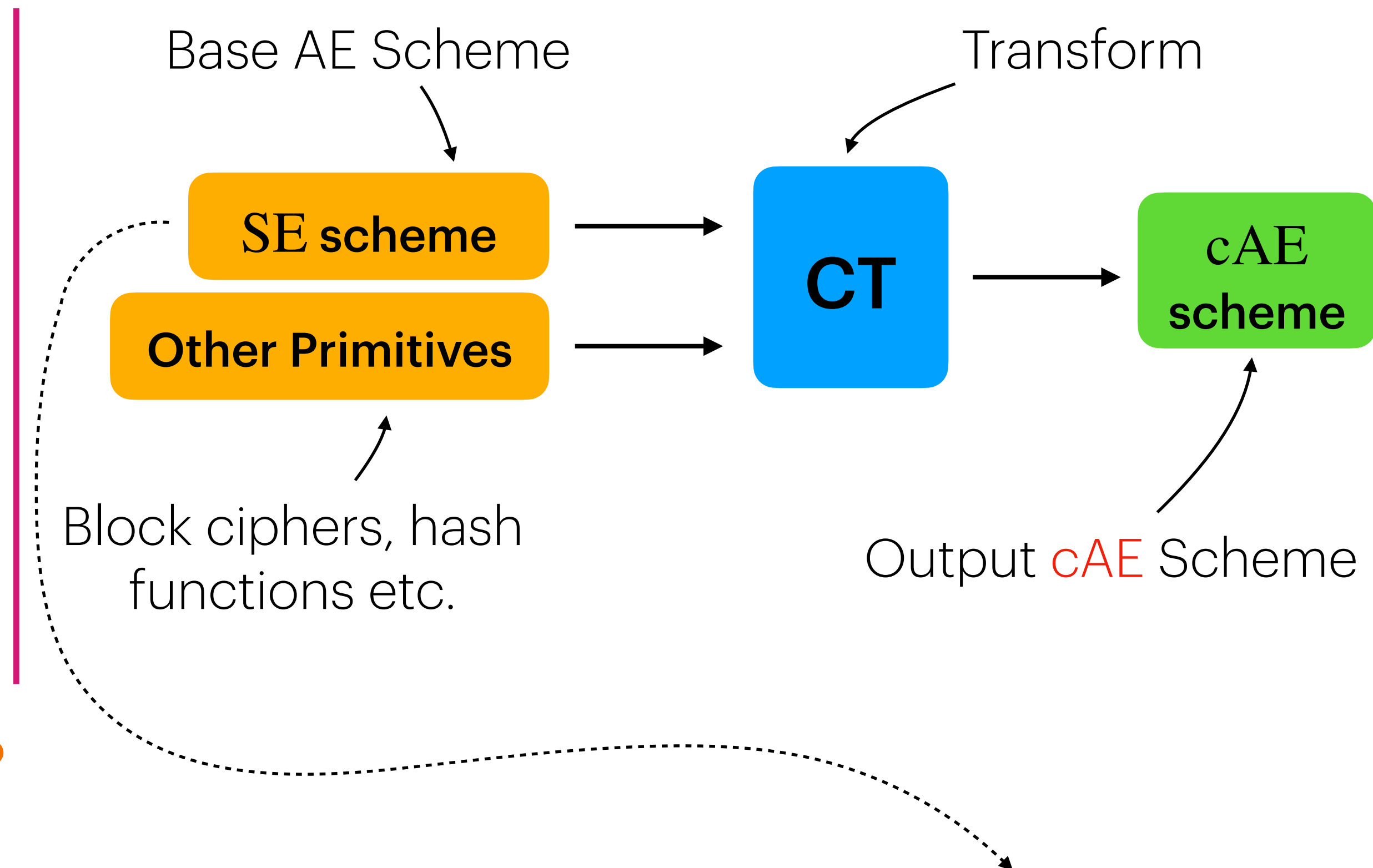
Many such transforms exist:

HtE ◦ UtC [BH22]
HtE ◦ RtC [BH22]
CTX [CR22]
CTY [BH24]
XtH [MBHLR25]
HtAE [CK25]
AEaH [CK25]
AEtH [CK25]

So what is the problem?

Each transform works **only for a sub-class** of all base AE schemes **SE**, with the sub-class differing from scheme to scheme

None of these transforms is **universal**, meaning **ALWAYS WORKs**.



How are cAE schemes constructed?

Committing AE schemes are constructed via **generic transforms** that

- **Preserve AE**: If SE is AE-secure, then so is cAE
- **Add committing security**: cAE is additionally CMT-secure

Many such transforms exist:

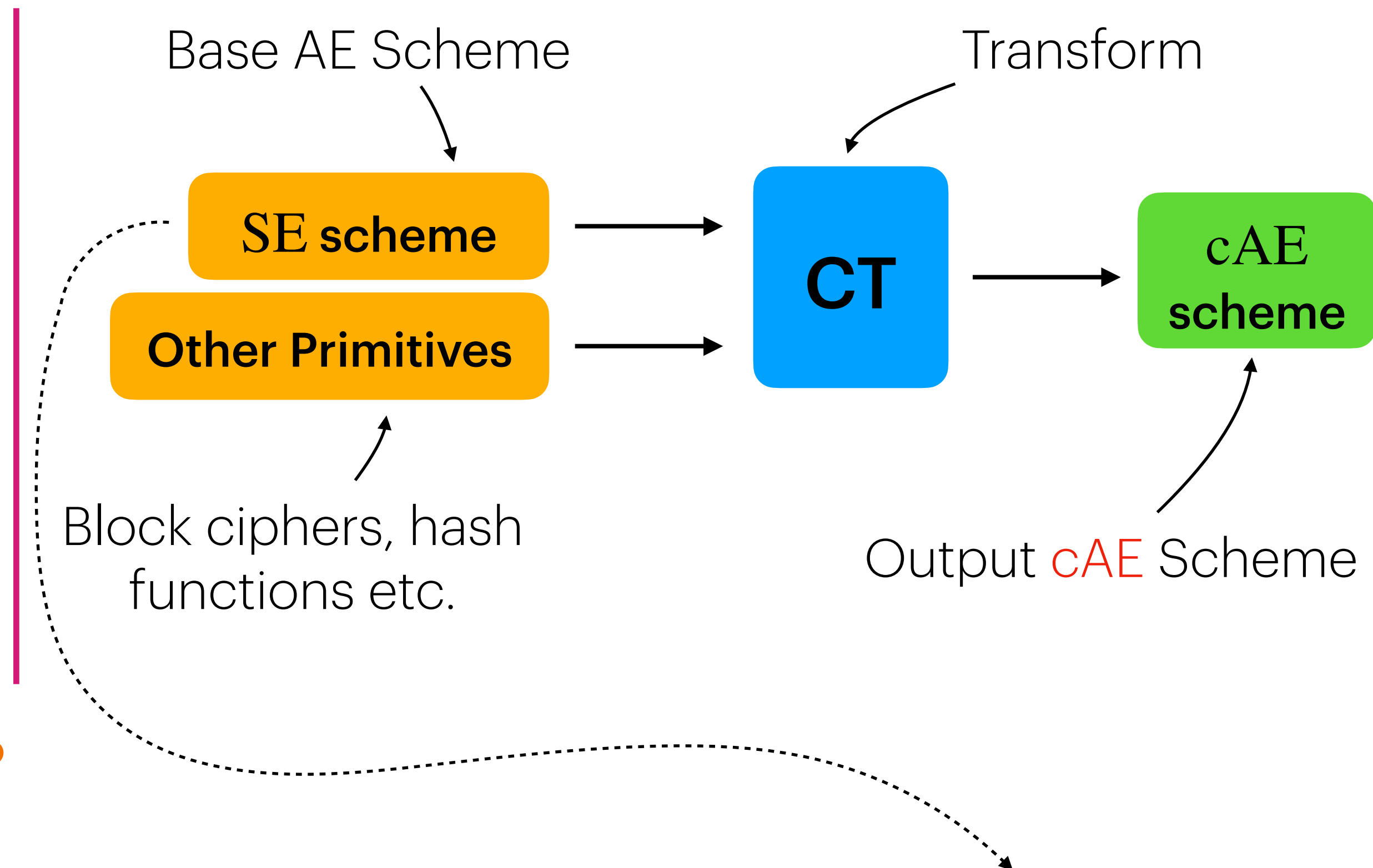
HtE ◦ UtC [BH22]
HtE ◦ RtC [BH22]
CTX [CR22]
CTY [BH24]
XtH [MBHLR25]
HtAE [CK25]
AEaH [CK25]
AEtH [CK25]

So what is the problem?

Each transform works **only for a sub-class** of all base AE schemes **SE**, with the sub-class differing from scheme to scheme

None of these transforms is **universal**, meaning **ALWAYS WORKs**.

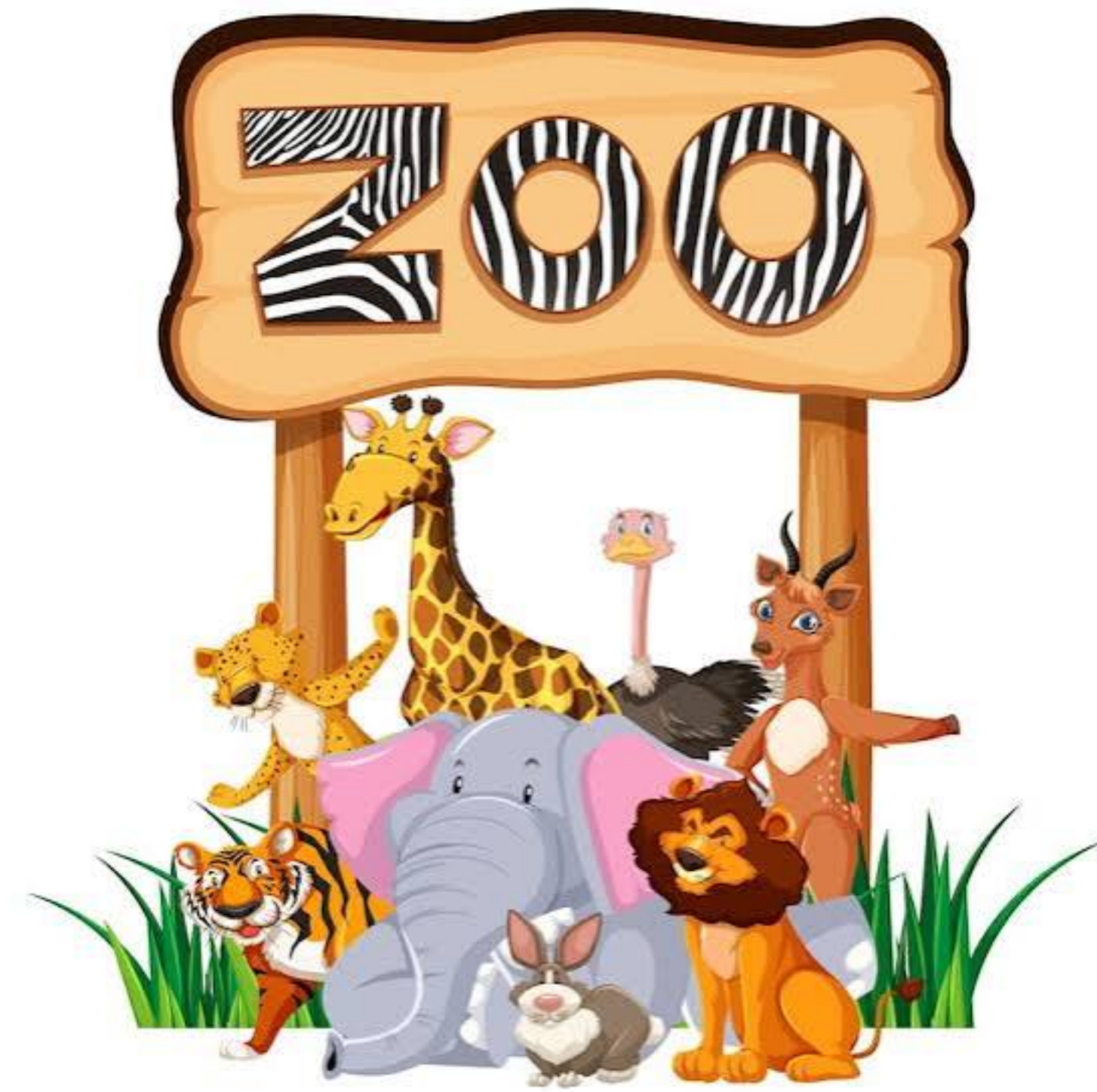
We ask if there is a **universal transform**
And give one, called **UCX**

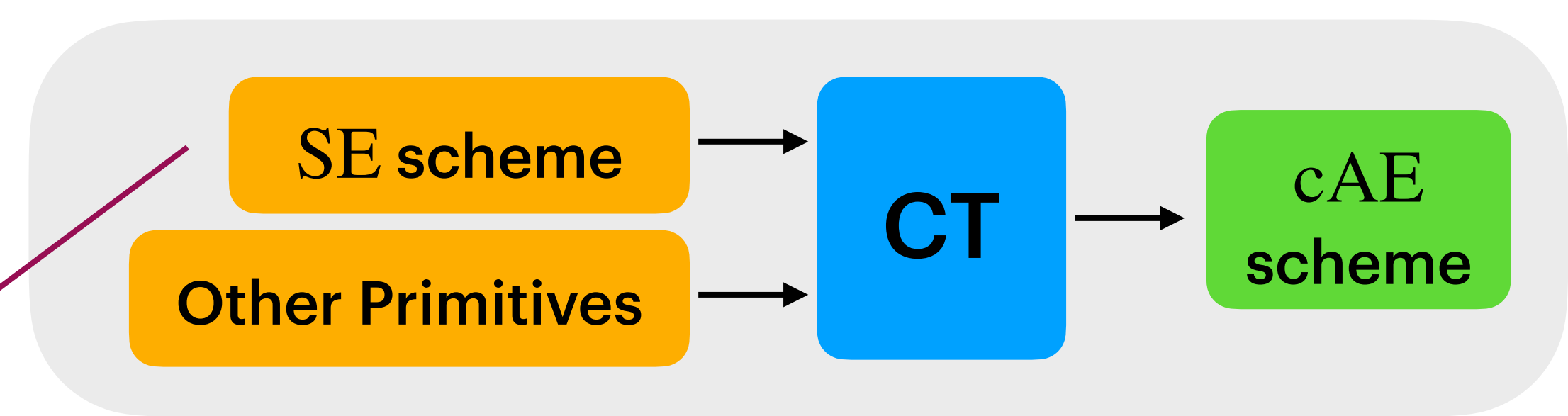
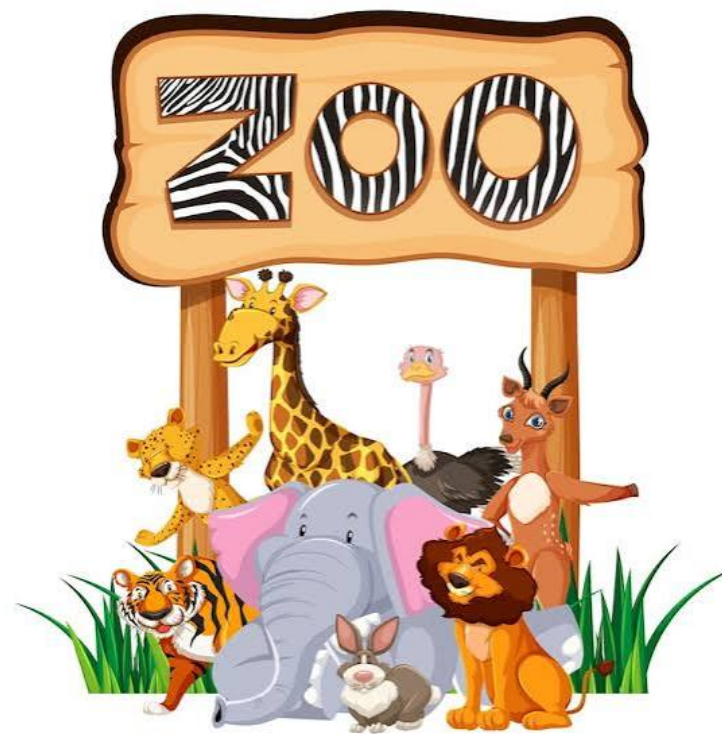


AE is not monolithic

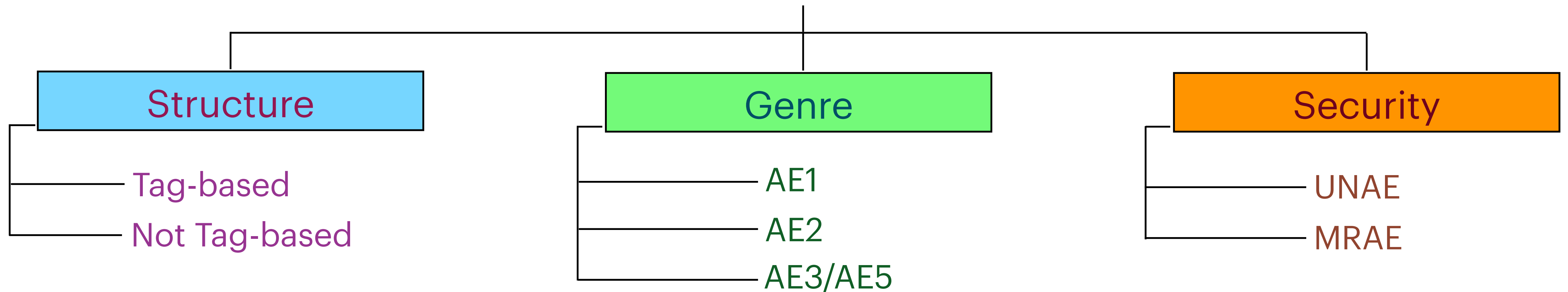
AE is not monolithic

It is a





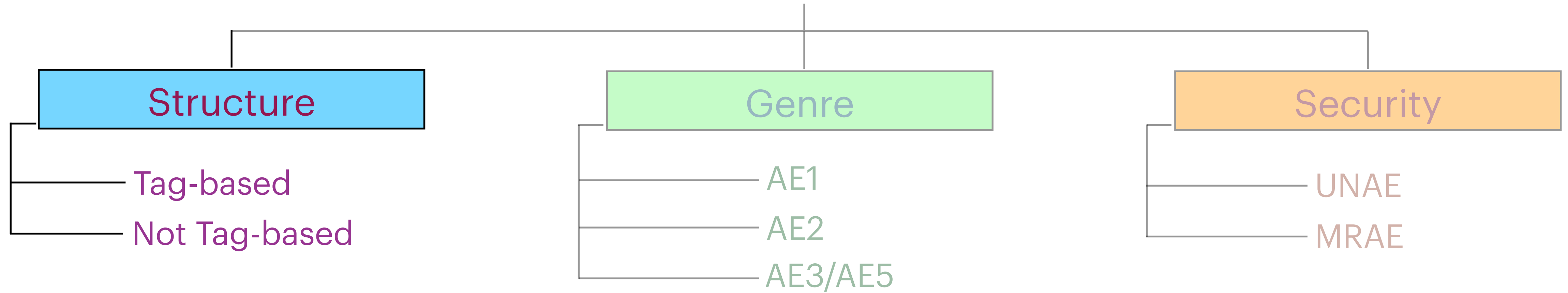
Base schemes SE differ in many ways:



Our requirement for a transform CT to be **universal** is that

It works across all the choices of SE in the zoo
And it is efficient

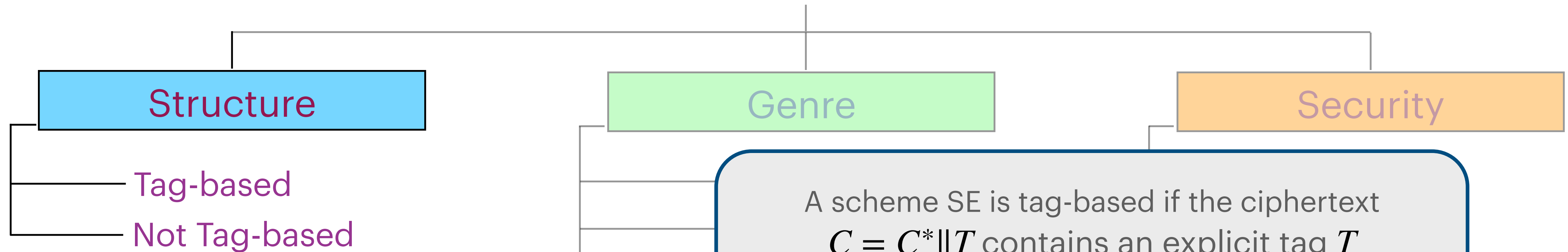
Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

- 1 No limitation on **structure** of base scheme

Base schemes SE differ in many ways:



Our requirements for a transform CT to be **uni**

1

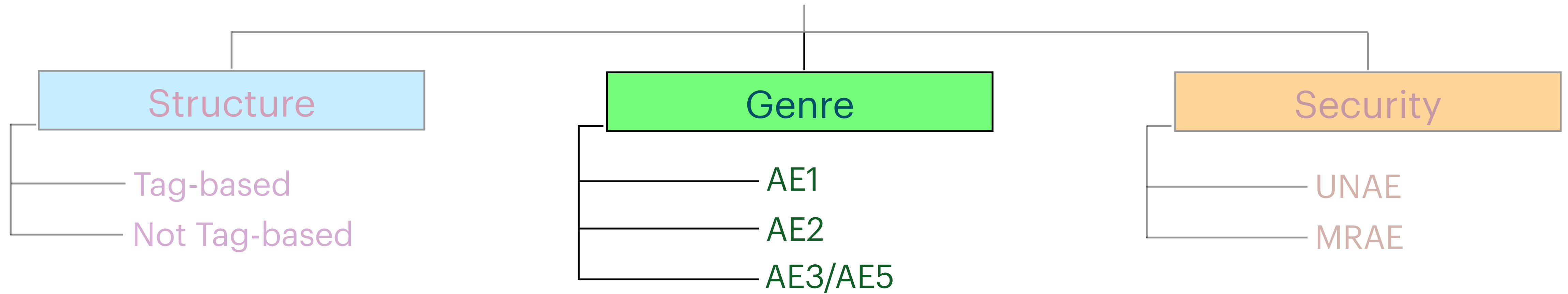
No limitation on **structure**
of base scheme

A scheme SE is tag-based if the ciphertext
 $C = C^* || T$ contains an explicit tag T
Decryption computes a candidate tag T^* from
 C^* and verifies that $T = T^*$

Prior transforms such as
CTX [CR22], CTY [BH24] and XtH [MBHLR25]
require the base scheme to be tag-based.

There are important not-tag-based schemes:
Schemes built using Encode-then-Encipher
paradigm (**AEZ [HKR15]**) and using the SIV method
(**AES-GCM-SIV [GLL19]**) are not-tag-based.

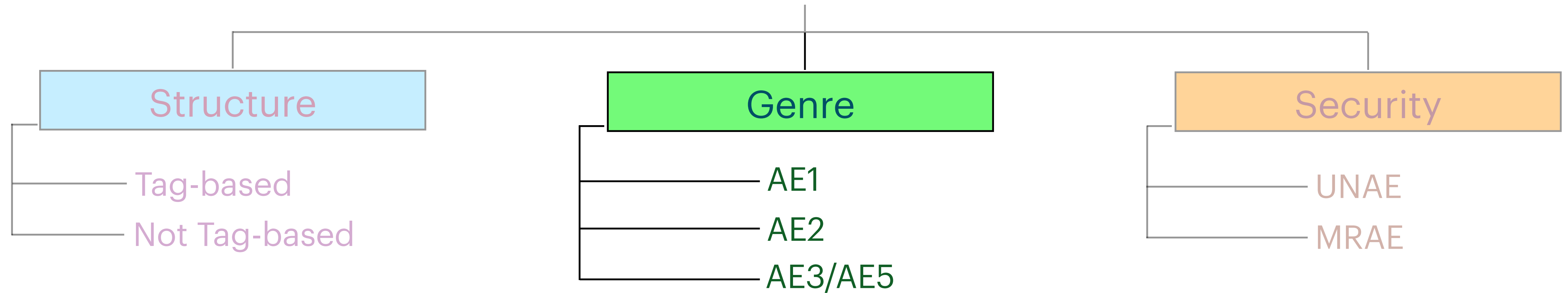
Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

- 1 No limitation on **structure** of base scheme
- 2 Works with **AE5** base schemes

Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

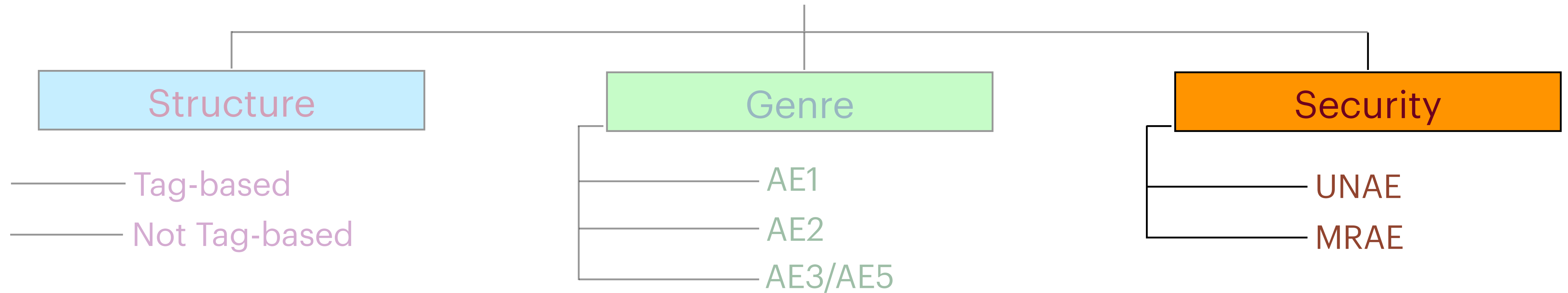
- 1 No limitation on **structure** of base scheme
- 2 Works with **AE5** base schemes

AE1 has fully public nonces which are **shown to lead to privacy issues** [BNT19].

AE2 [BNT19] has fully private nonces to remedy this but **is restrictive**.

AE3, AE5 [BS21, NRS13] allow both public and private nonces. They **captures AE1 and AE2 as special cases**. Standards should move towards AE5 adoption!

Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

1

No limitation on **structure**
of base scheme

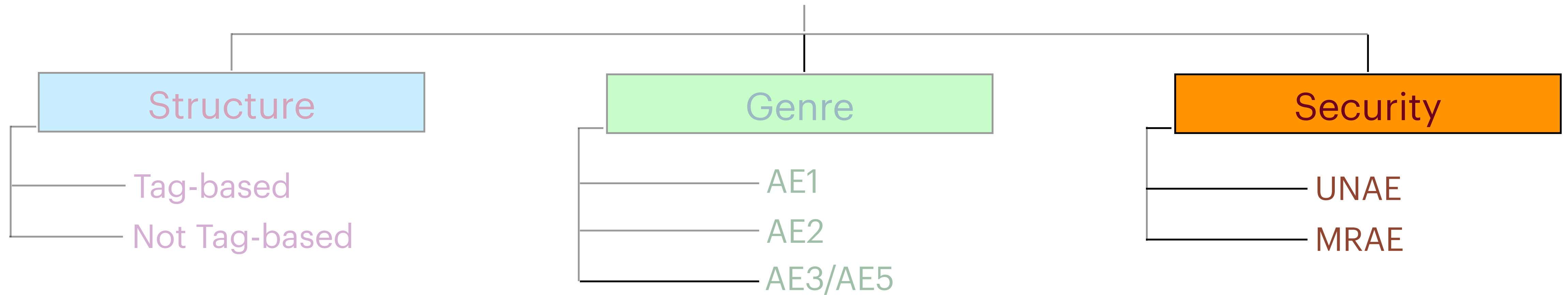
2

Works with **AE5** base
schemes

3

Preserves both **UNAE**
and **MRAE** security

Base schemes SE differ in many ways:



UNAE = Unique Nonce AE, security assuming nonces do not repeat.
MRAE = Misuse-resistant AE, security even if nonces repeat.

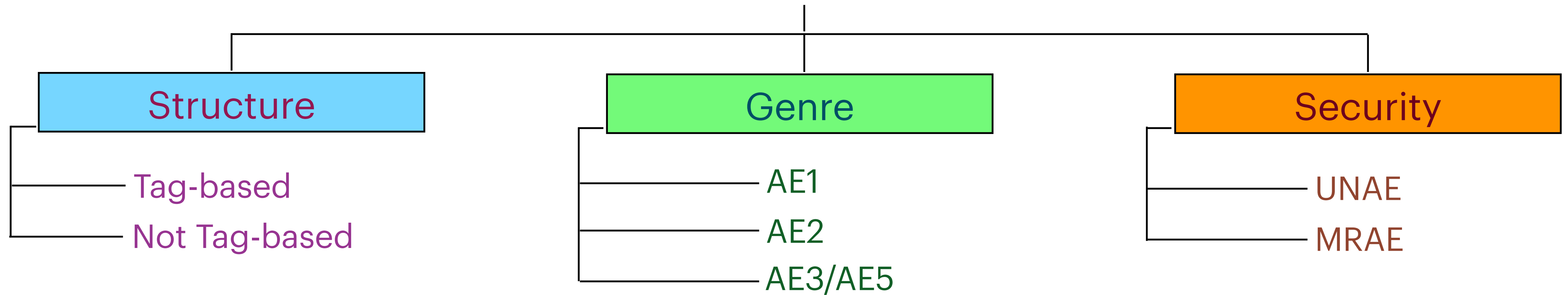
The transform should be able to
add committing security to both UNAE and MRAE secure schemes while preserving them, respectively.

Meaning, if the base scheme is XXAE secure then the transformed scheme should also be XXAE secure ($XX \in \{UN, MR\}$).

3

Preserves both **UNAE**
and **MRAE** security

Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

- 1 No limitation on **structure** of base scheme
- 2 Works with **AE5** base schemes
- 3 Preserves both **UNAE** and **MRAE** security

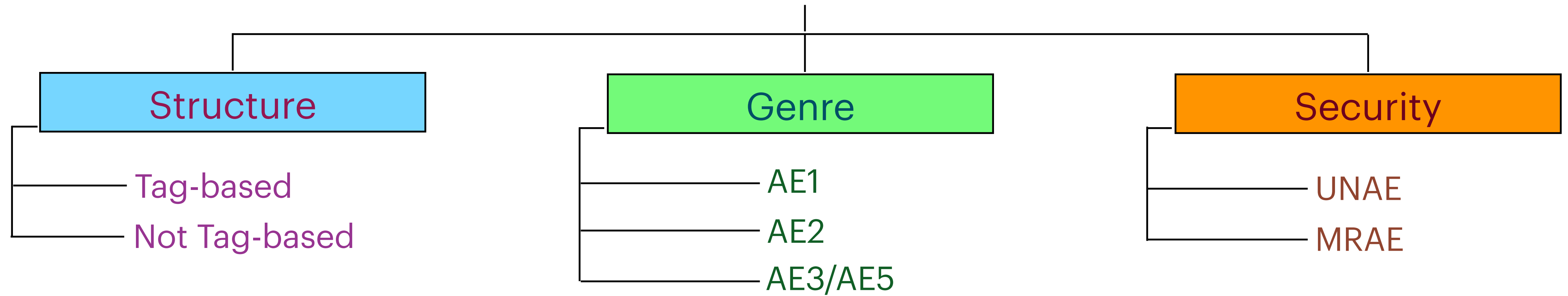
- 4 Is **efficient** in computation and bandwidth

Computational overhead should be independent of message length.

For s bits of committing security, ciphertext overhead $co(s)$ = length of ciphertext in transformed (cAE) scheme minus length of ciphertext in base SE scheme should be $\leq s$.

Want $s = 128$ so want ciphertext expansion ≤ 128 bits.

Base schemes SE differ in many ways:



Our requirements for a transform CT to be **universal** :

- 1 No limitation on **structure** of base scheme
- 2 Works with **AE5** base schemes
- 3 Preserves both **UNAE** and **MRAE** security
- 4 Is **efficient** in computation and bandwidth

But, WHY?

Why do we want a universal transform?

What's wrong with different, dedicated transforms for different classes of base schemes?

Some reasons to find a nice universal transform:

1. Theoretical: Clean, general, single solution.

2. Practical: Easy to use, resistant to misuse.

An implementor may find it hard to figure out which dedicated transform to use with their base scheme, and may get it wrong. These difficulties are avoided with a universal transform.

3. Prior work: The question of finding a universal transform was raised and pursued in prior work [BBD24].

Prior transforms at not universal

Transform	1 Base AE Tag-based?	2 Base AE Type	3 Transform Preserves	4 co(128)
HtE ◦ UtC [BH22]	No	AE1	UNAE	256
HtE ◦ RtC [BH22]	No	AE1	UNAE, MRAE	256
CTX [CR22]	Yes	AE1	UNAE	128
CTY [BH24]	Yes	AE5	UNAE	128
XtH [MBHLR25]	Yes	AE5	UNAE	128
HtAE [CK25]	No	AE1	UNAE	256
AEaH [CK25]	No	AE1	UNAE	256
AEtH [CK25]	No	AE1	UNAE, MRAE	256

Prior transforms are not universal

Transform	1 Base AE Tag-based?	2 Base AE Type	3 Transform Preserves	4 co(128)
HtE ◦ UtC [BH22]	No	AE1	UNAE	256
HtE ◦ RtC [BH22]	No	AE1	UNAE, MRAE	256
CTX [CR22]	Yes	AE1	UNAE	128
CTY [BH24]	Yes	AE5	UNAE	128
XtH [MBHLR25]	Yes	AE5	UNAE	128
HtAE [CK25]	No	AE1	UNAE	256
AEaH [CK25]	No	AE1	UNAE	256
AEtH [CK25]	No	AE1	UNAE, MRAE	256

PACT and comPACT [BBD24] are two other transforms adding committing security. They require a non-standard property, *ciphertext collision* on the encryption scheme.
We give attacks to show that a common base scheme like GCM does not satisfy this property.

Transform	1 Base AE Tag-based?	2 Base AE Type	3 Transform Preserves	4 co(128)
HtE ◦ UtC [BH22]	No	AE1	UNAE	256
HtE ◦ RtC [BH22]	No	AE1	UNAE, MRAE	256
CTX [CR22]	Yes	AE1	UNAE	128
CTY [BH24]	Yes	AE5	UNAE	128
XtH [MBHLR25]	Yes	AE5	UNAE	128
HtAE [CK25]	No	AE1	UNAE	256
AEaH [CK25]	No	AE1	UNAE	256
AEtH [CK25]	No	AE1	UNAE, MRAE	256
UCX [Ours]	No	AE5	UNAE, MRAE	128



Our Contributions

✓ New Primitive: Tweakable Committing Concealer (TCC)

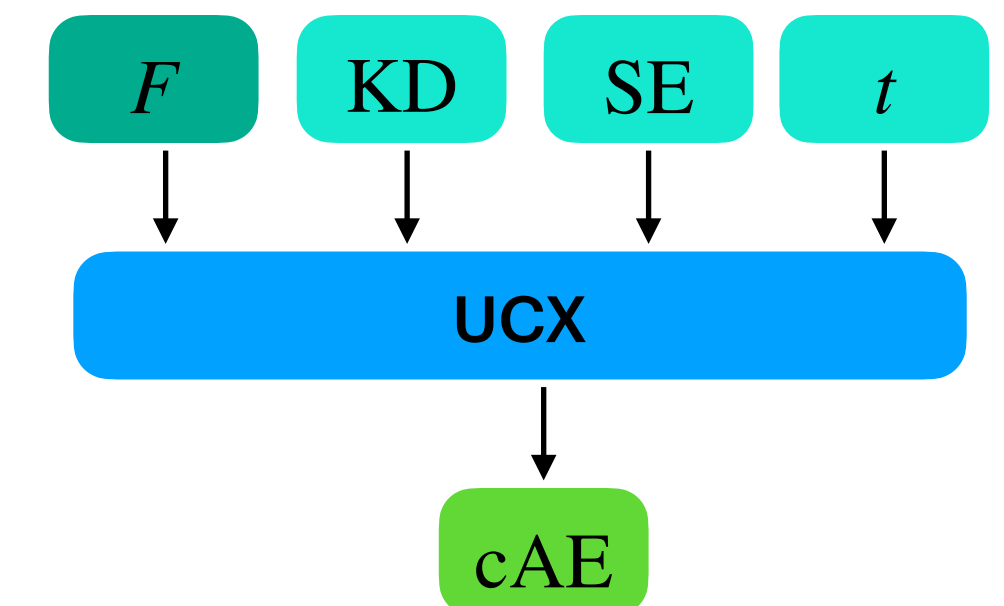
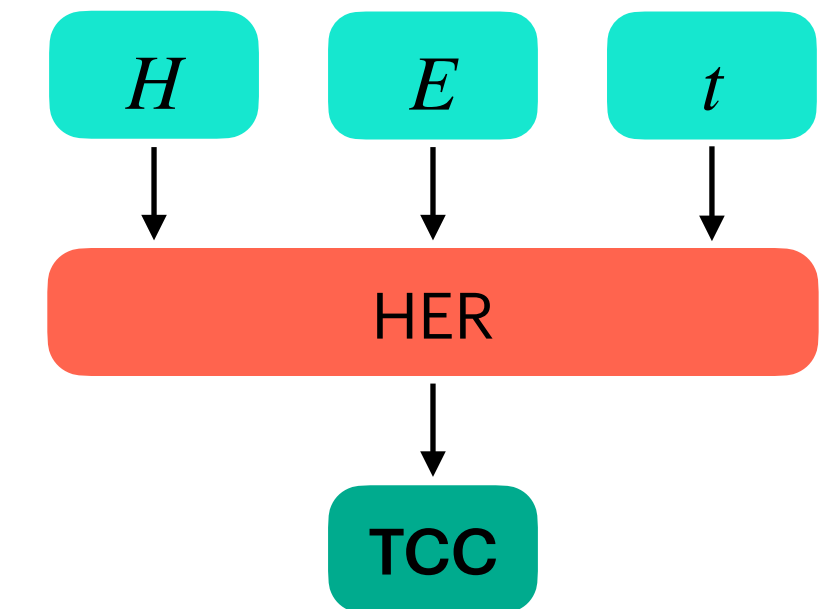
- Expands on Committing Concealers [HM24].
- Used as a tool in our universal transform.

✓ HER transform to construct a TCC

- Constructs a TCC F from a keyed hash function H , block cipher E and integer t .
- We show **Collision Resistance of F** in the **ideal cipher model** assuming **collision resistance of H** .
- We show that F is a **Pseudorandom Injection** (PRI) if H is a PRF and E is a strong PRP. No idealized assumptions needed.

✓ A universal Committing transform: UCX

- Constructs a cAE scheme from a TCC F , key derivation function **KD**, AE5 scheme **SE** and integer t . The output scheme is also AE5.
- We show **UCX provides committing security** and **preserves UNAE and MRAE security**.

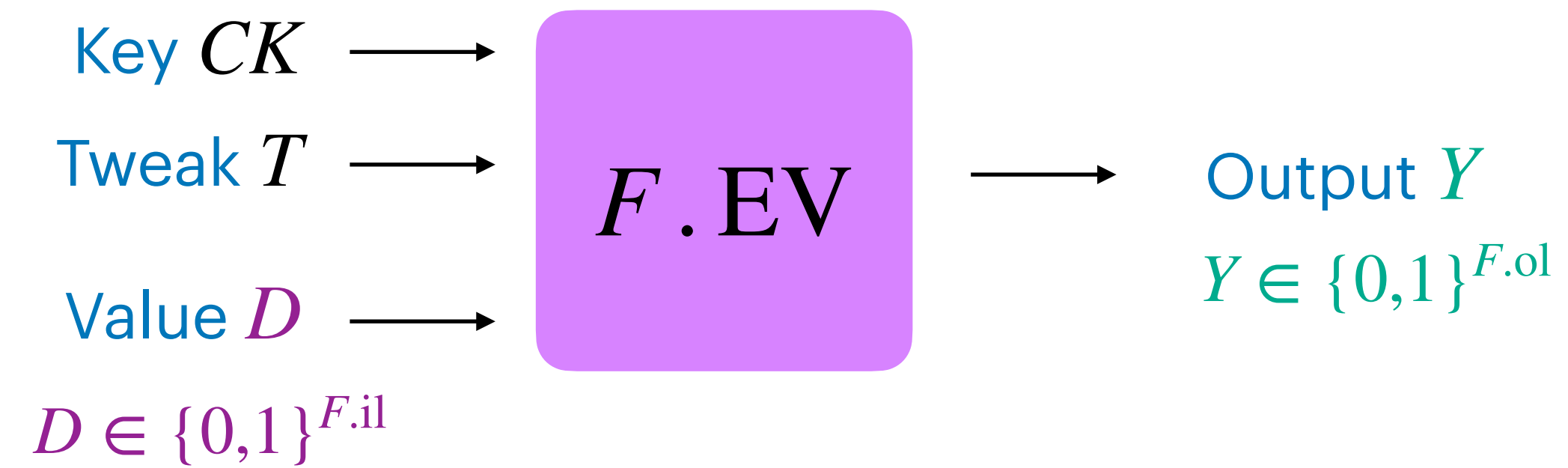


Tweakable Committing Concealers

A function family with partial invertibility.

Definition of a TCC F

• Evaluation



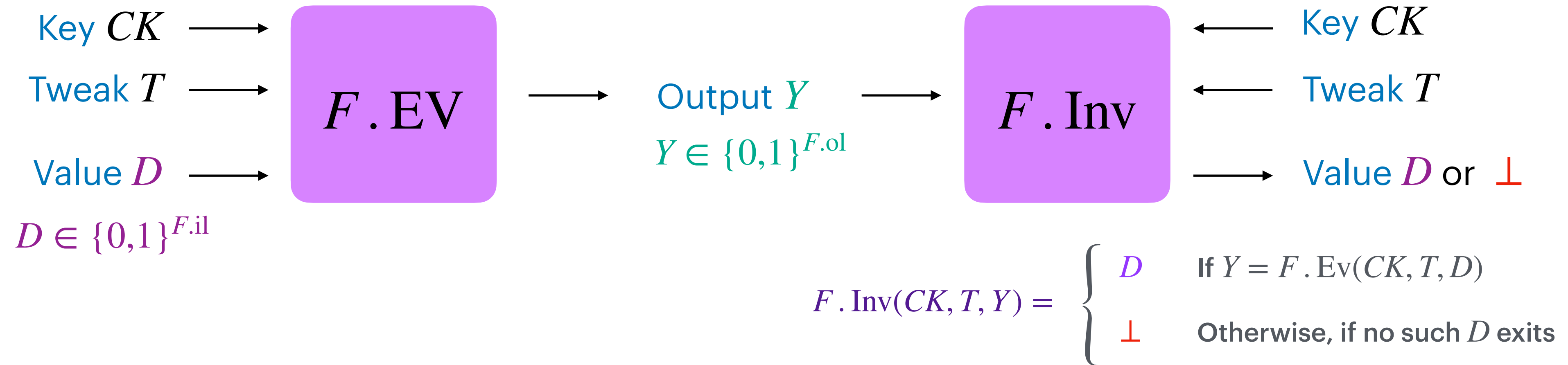
Tweakable Committing Concealers

A function family with partial invertibility.

Definition of a TCC F

• Evaluation

• Inverse



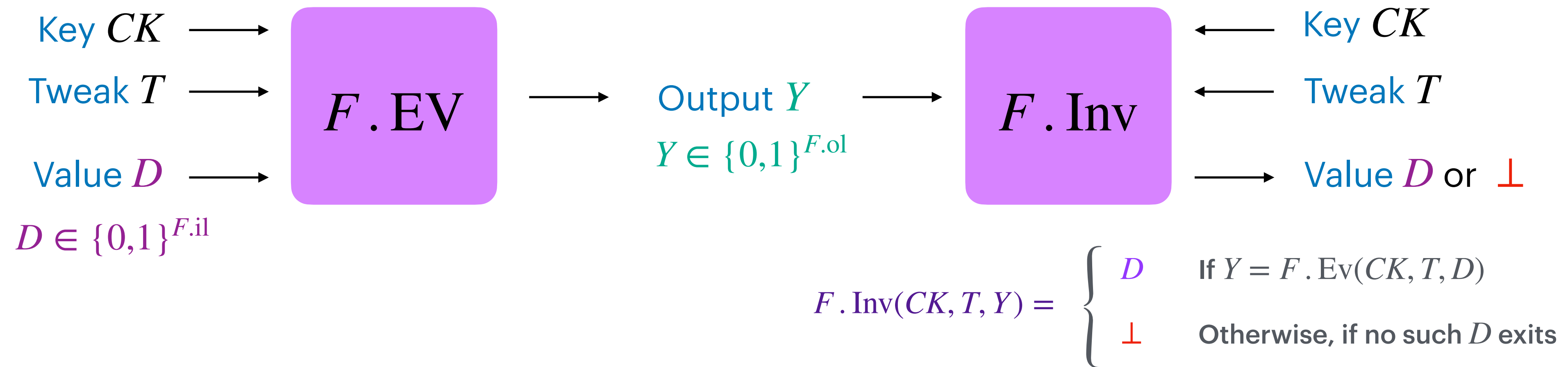
Tweakable Committing Concealers

A function family with partial invertibility.

Definition of a TCC F

• Evaluation

• Inverse



Security

- **Collision Resistance** of $F.Ev$.
- **Pseudorandom Injection:** this requires that F for any (hidden) key and tweak is indistinguishable from random injection.

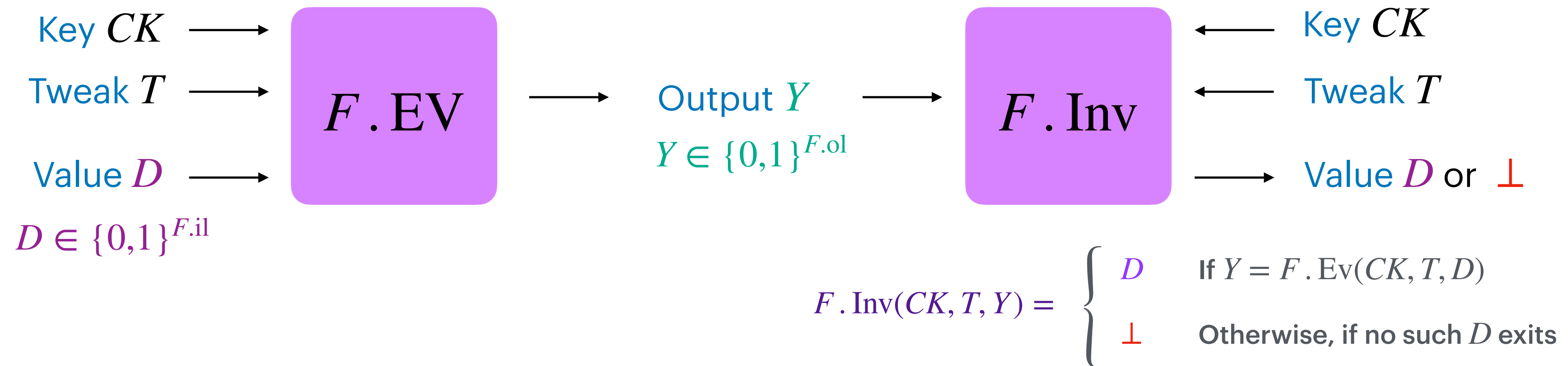
Tweakable Committing Concealers

A function family with partial invertibility.

Definition of a TCC F

• Evaluation

• Inverse



Security

- **Collision Resistance** of $F.EV$.
- **Pseudorandom Injection**: this requires that F for any (hidden) key and tweak is indistinguishable from random injection.

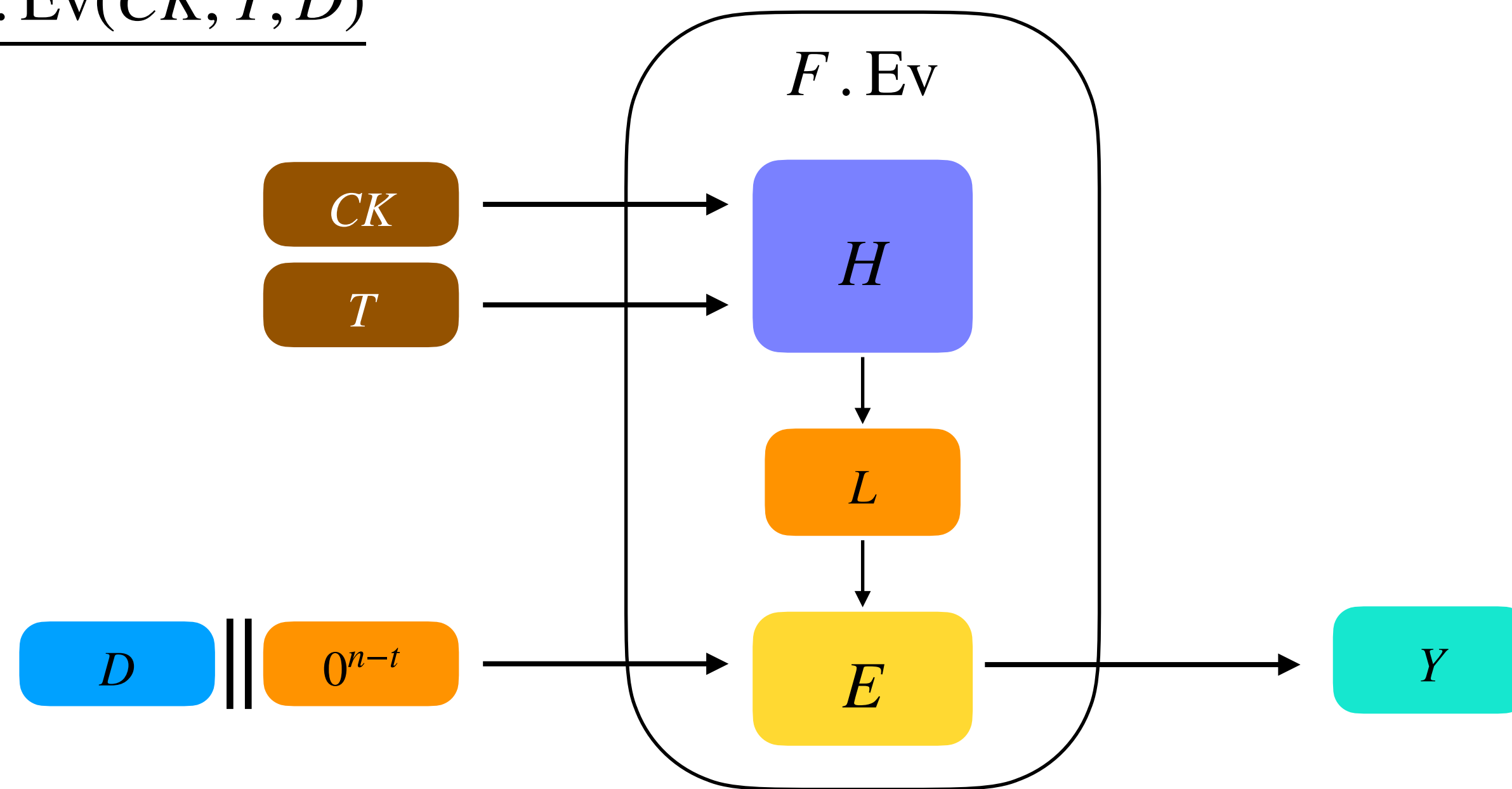
Relation to other primitives

- **Tweakable Block Cipher**: Syntactically same as TCC if $F.il = F.ol$. But TBCs are not collision resistant.
- **Committing Concealer**: These are block ciphers without tweaks and so different from TCCs. The security properties considered are different as well.
- **Invertible PRF**: These do not support tweaks. Desired security is different.

HER Transform and Results

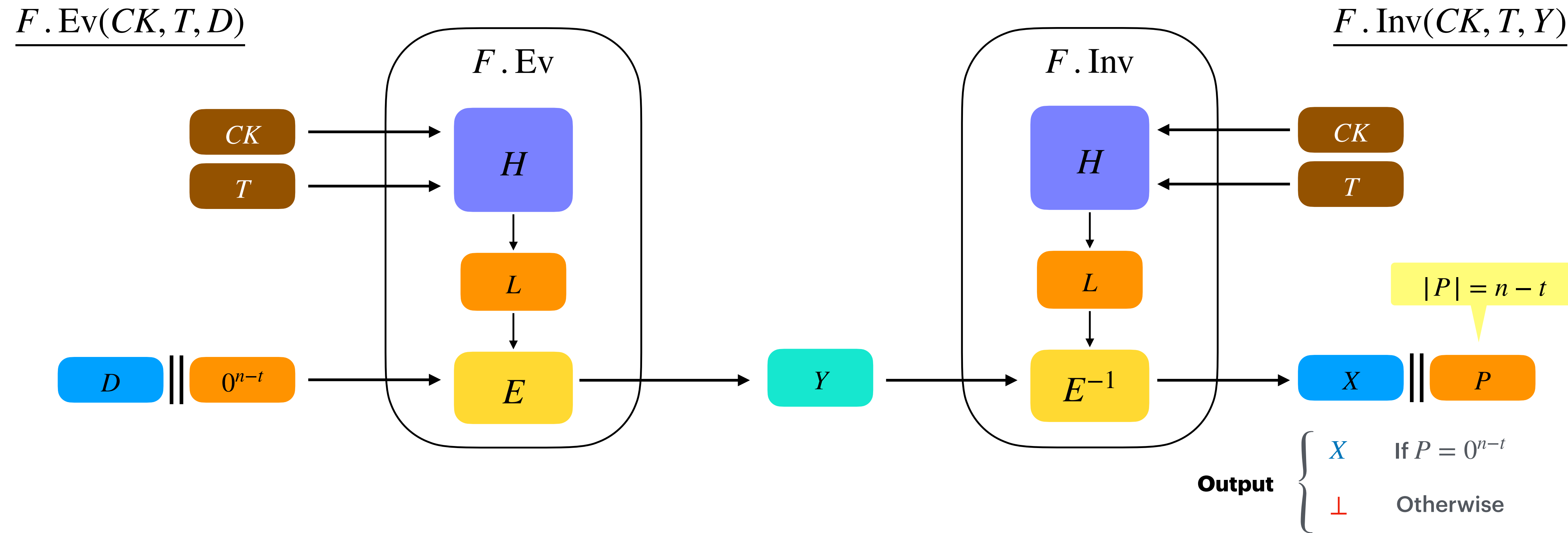
Given a hash function $H: \{0,1\}^k \times \{0,1\}^* \rightarrow \{0,1\}^\ell$,
a block cipher $E: \{0,1\}^\ell \times \{0,1\}^n \rightarrow \{0,1\}^n$ and an integer t
HER outputs a TCC F such that

$F.\text{Ev}(CK, T, D)$



HER Transform and Results

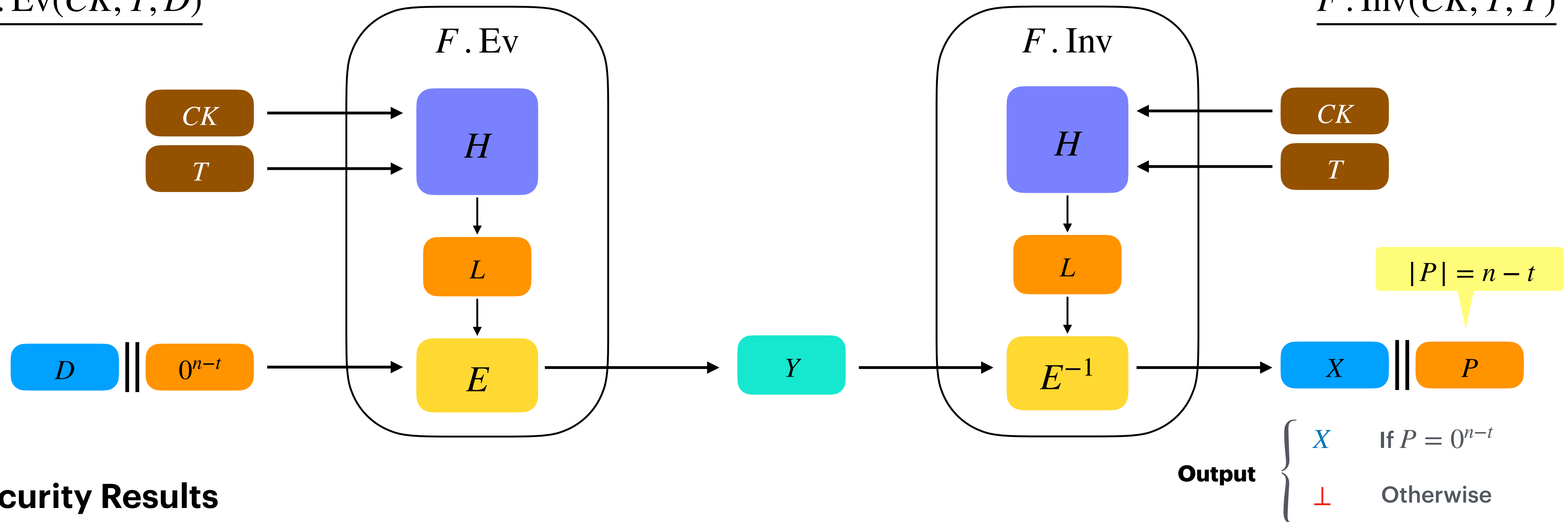
Given a hash function $H: \{0,1\}^k \times \{0,1\}^* \rightarrow \{0,1\}^\ell$,
a block cipher $E: \{0,1\}^\ell \times \{0,1\}^n \rightarrow \{0,1\}^n$ and an integer t
HER outputs a TCC F such that



HER Transform and Results

Given a hash function $H: \{0,1\}^k \times \{0,1\}^* \rightarrow \{0,1\}^\ell$,
a block cipher $E: \{0,1\}^\ell \times \{0,1\}^n \rightarrow \{0,1\}^n$ and an integer t
HER outputs a TCC F such that

$F . \text{Ev}(CK, T, D)$



Security Results

Collision Resistance

- E is assumed to be a ideal-cipher.
- Tight reduction to CR of H .

Pseudorandom Injection

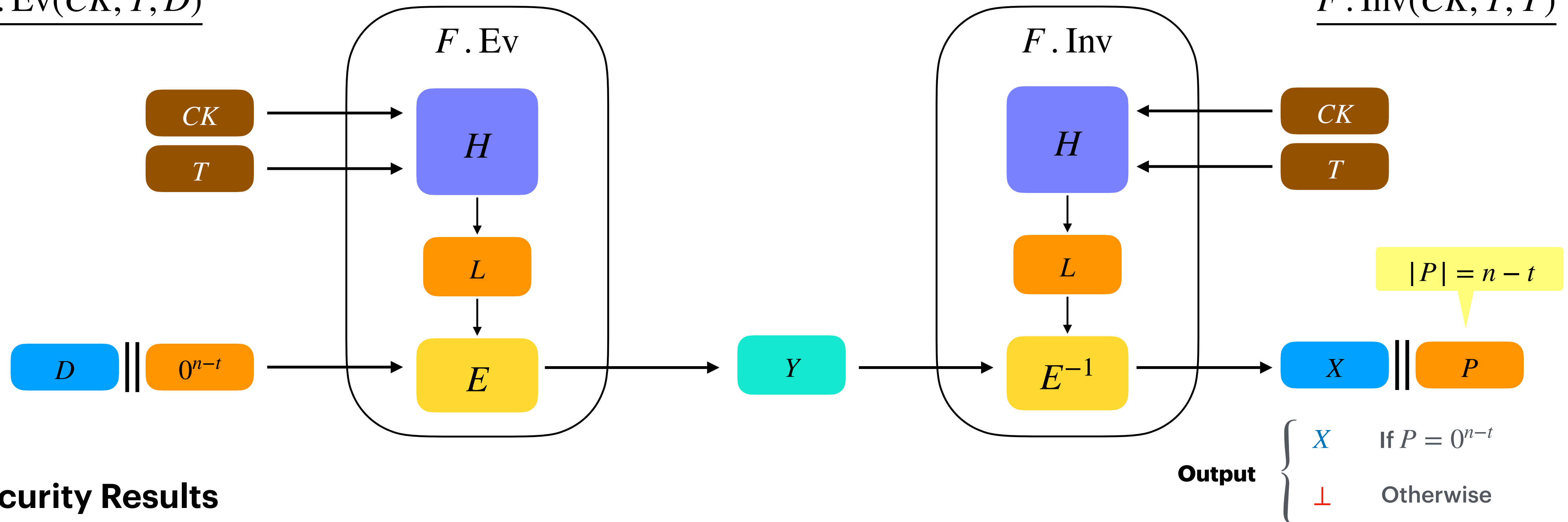
- Tight reduction to PRF security of H and strong-PRP security of E
- No idealized assumptions needed.

HER Transform and Results

Given a hash function $H: \{0,1\}^k \times \{0,1\}^* \rightarrow \{0,1\}^\ell$,
a block cipher $E: \{0,1\}^\ell \times \{0,1\}^n \rightarrow \{0,1\}^n$ and an integer t
HER outputs a TCC F such that

For 128-bit CR and PRI security,
 E could be instantiated with **Rijndael-256** and
 H could be instantiated with **truncated HMAC-SHA512**.

$F . \text{Ev}(CK, T, D)$



Security Results

Collision Resistance ✓

- E is assumed to be a ideal-cipher.
- Tight reduction to CR of H .

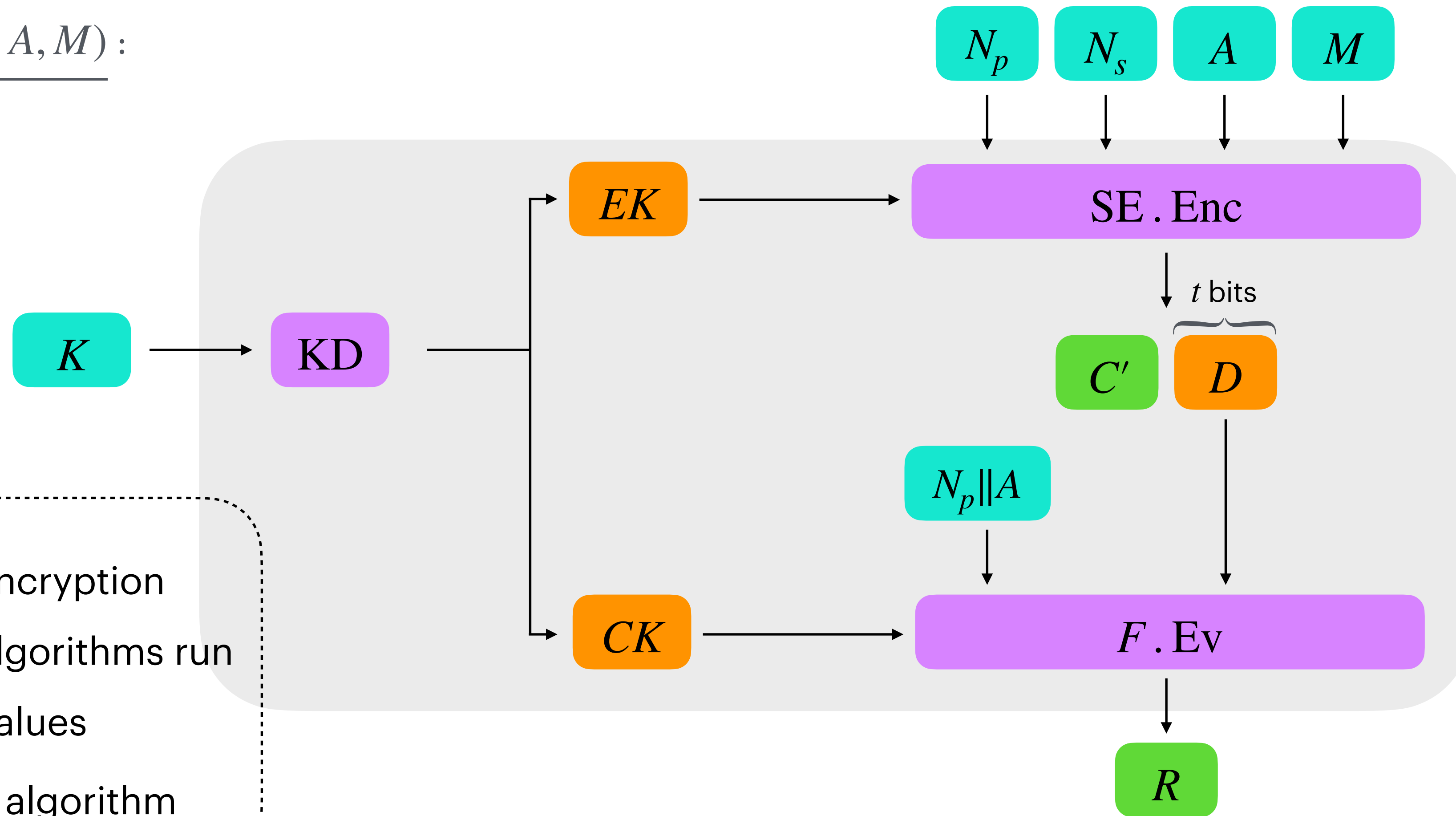
Pseudorandom Injection ✓

- Tight reduction to PRF security of H and strong-PRP security of E
- No idealized assumptions needed.

UCX Transform

Given a TCC F , a Key Derivation Function KD , an AE5 scheme SE and an integer t
UCX outputs a committing AE5 scheme $\overline{SE}$ such that

$\overline{SE} . \text{Enc}(K, N_p, N_s, A, M) :$



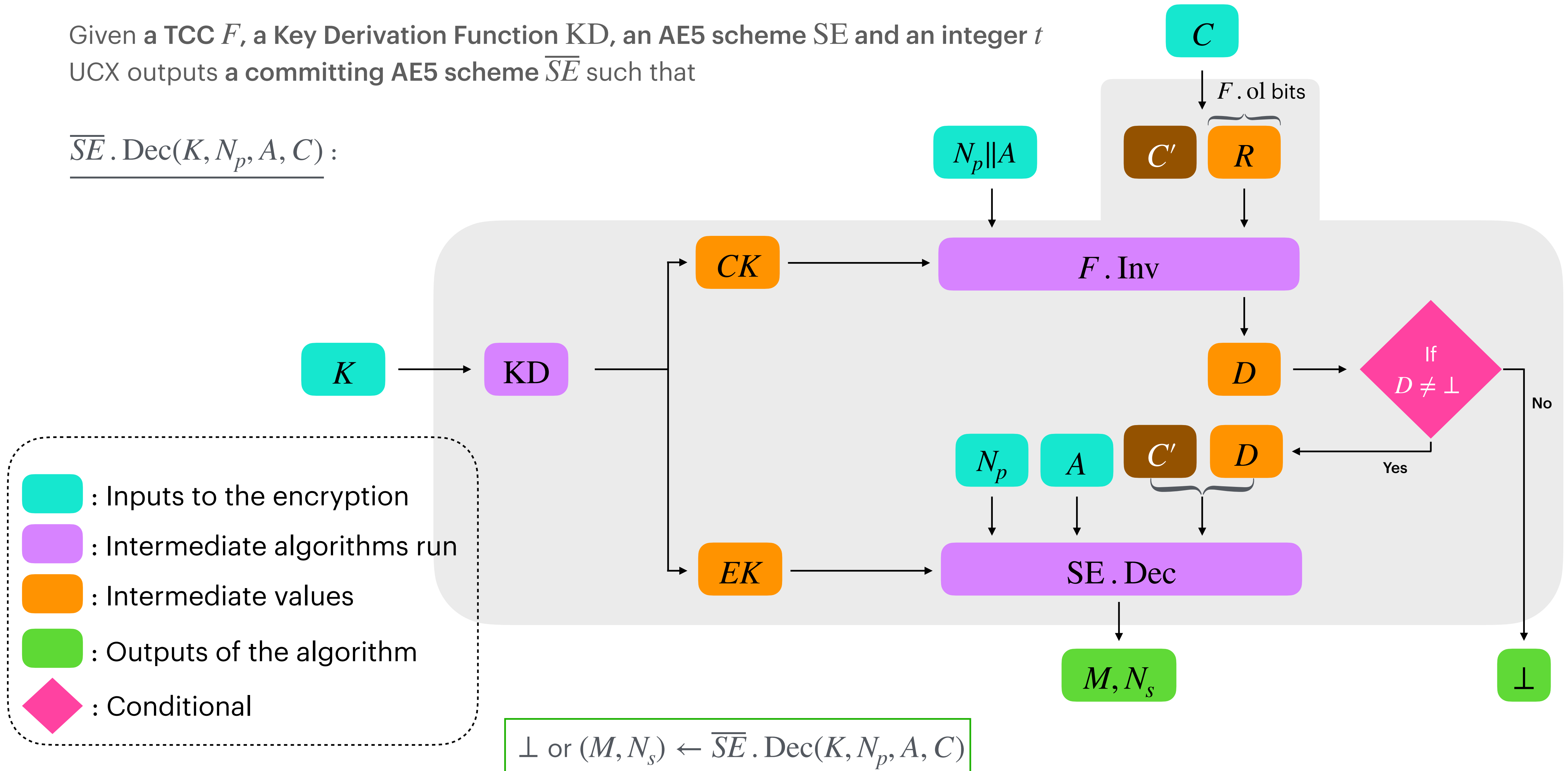
- : Inputs to the encryption
- : Intermediate algorithms run
- : Intermediate values
- : Outputs of the algorithm

$$C' || R \leftarrow \overline{SE} . \text{Enc}(K, N_p, N_s, A, M)$$

UCX Transform

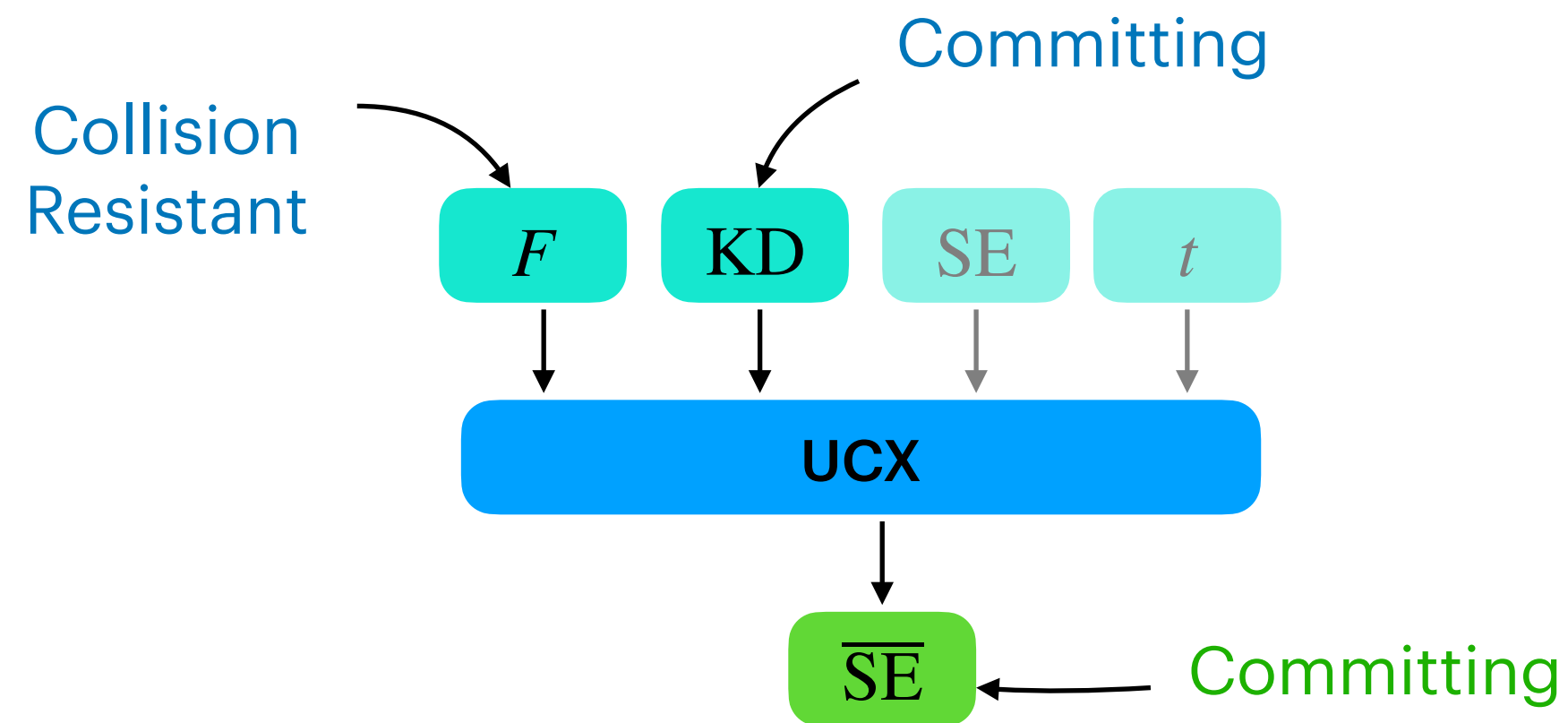
Given a TCC F , a Key Derivation Function KD , an AE5 scheme SE and an integer t
 UCX outputs a committing AE5 scheme $\overline{SE}$ such that

$\overline{SE} . Dec(K, N_p, A, C) :$

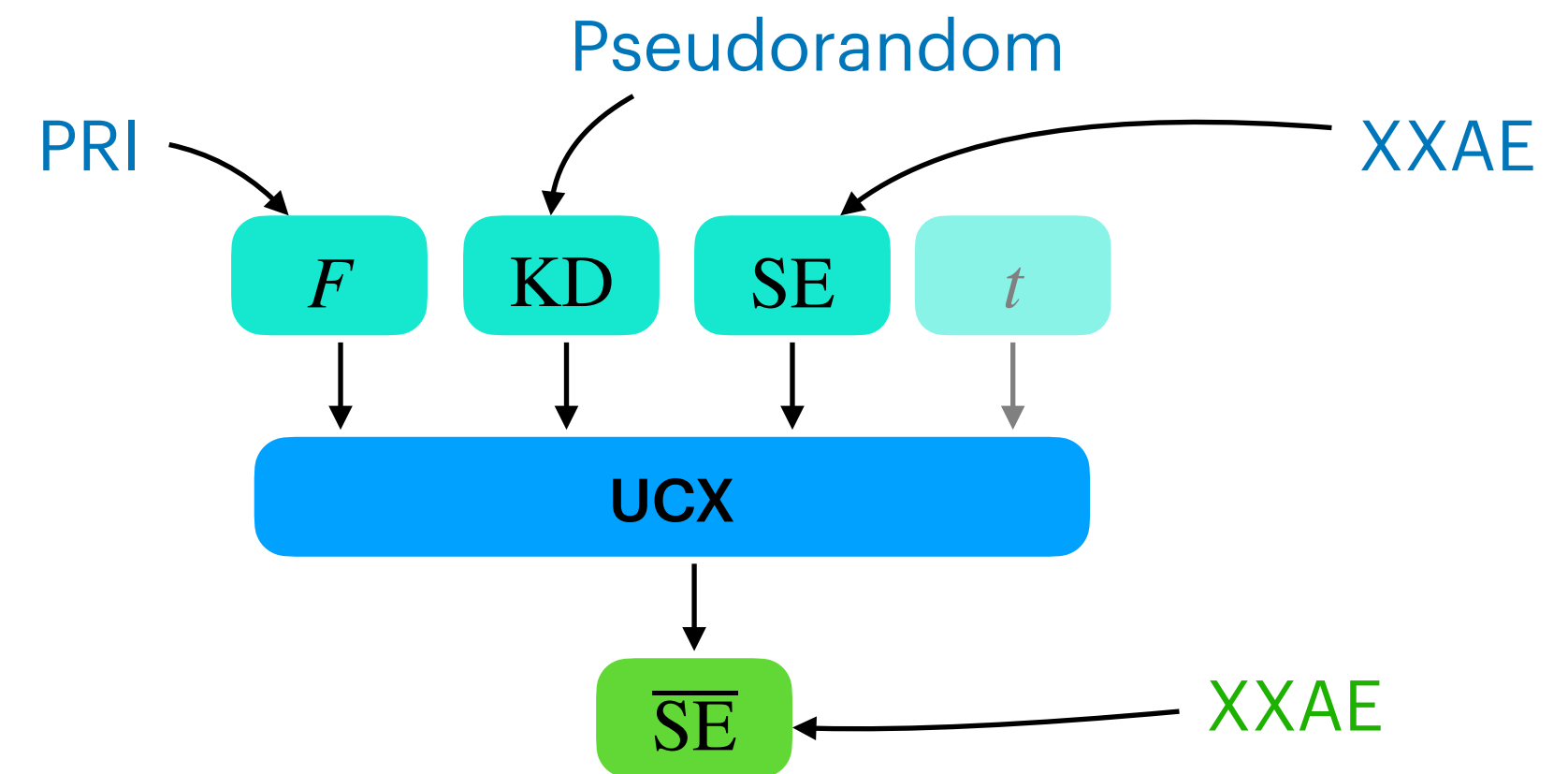


UCX Transform: Security Results

◎ **UCX provides commitment security:**



◎ **UCX preserves UNAE and MRAE security:**



Some Remarks

- ◎ **UCX preserves tag-basedness:** If the underlying AE5 scheme is tag-based, then output scheme of UCX is also tag-based. Thus, ciphertext shortening transform SC [BH24] can be applied if the underlying scheme is tag-based to reduce commitment ciphertext overhead.
- ◎ **We can further optimize UCX if only UNAE security is required to be preserved.** The idea is similar to the optimization done in CTY [BH24] where associated data is not input to the base scheme encryption. This does not preserve MRAE security which is broken with this optimization.

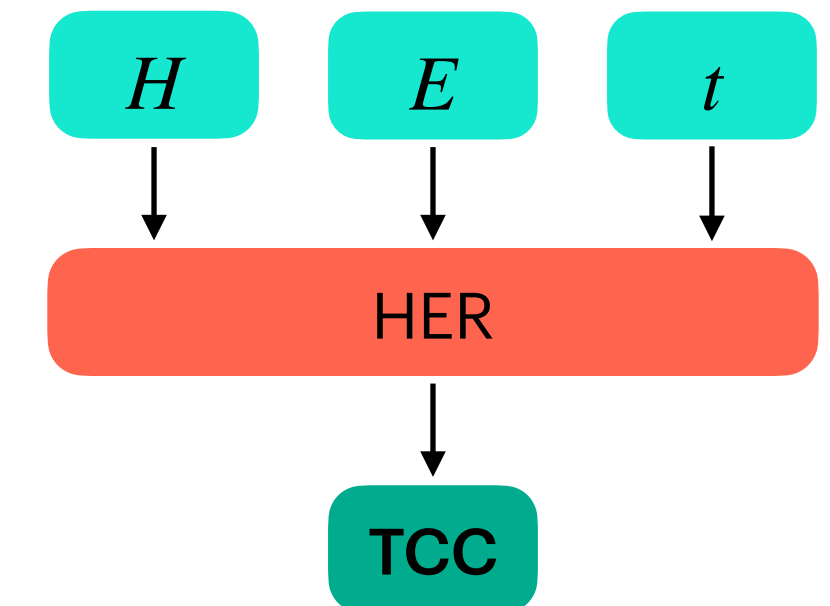
Summary

☑ New Primitive: Tweakable Committing Concealer (TCC)

- Expands on Committing Concealers [HM24].
- Used as a tool to in our universal transform.

☑ HER transform to construct a TCC

- Constructs a TCC F from a keyed hash function H and a block cipher E .
- We show Collision Resistance of F in the ideal cipher model assuming collision resistance of H .
- We show F is a Pseudorandom Injection (PRI) if H is a PRF and E is a strong PRP. No idealized assumptions needed.



☑ A universal Committing AE transform: UCX

- Constructs a cAE scheme from a TCC F , key derivation function $\mathbf{KD}$, AE5 scheme $\mathbf{SE}$ and integer t . The output scheme is also AE5.
- We show UCX provides committing security and preserves UNAE and MRAE security.

